Rapport : Mise en place d'une architecture DMZ via GNS3 et Docker

Membres du groupe :

- 1979
- 2075

Introduction

L'objectif de ce travail pratique est de mettre en œuvre une architecture réseau sécurisée intégrant une zone démilitarisée (DMZ), en utilisant les outils GNS3 et Docker. Ce projet s'inscrit dans le cadre de l'apprentissage des techniques de défense périmétrique d'un système d'information. Il permet de concevoir, configurer et analyser une topologie réseau complète, comprenant la mise en place d'une DMZ, la gestion des règles de filtrage et de NAT, le déploiement de solutions de détection/prévention d'intrusion ainsi que l'analyse des logs pour valider la politique de sécurité. À travers la réalisation de ce TP, les compétences en conception d'architectures sécurisées, en administration réseau et en cybersécurité seront renforcées.

Méthodologie

La méthodologie suivie pour la réalisation de ce travail pratique s'articule autour des étapes suivantes

1. **Création des images Docker**

La première étape a consisté à créer les images Docker nécessaires pour simuler l'ensemble des machines de notre architecture DMZ. Plusieurs Dockerfiles ont été développés, chacun correspondant à un rôle spécifique dans l'infrastructure :

1.1 Client LAN (`ubuntu_client`)

Cette image simule une machine cliente dans le réseau LAN interne de l'entreprise :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    lynx \
    iputils-ping \
    traceroute \
    curl \
    wget \
    nano \
    openssh-client \
    iproute2 \
    dnsutils
```

Rôle : Machine cliente utilisée pour tester l'accès depuis le LAN vers la DMZ et Internet, et valider les règles de filtrage du pare-feu.

1.2 Pare-feu (`ubuntu_firewall`)

Cette image centralise toutes les fonctions de sécurité réseau :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    iptables \
    iproute2 \
    tcpdump \
    squid \
    squidguard \
    squid-cgi \
    apache2-utils \
    dnsutils \
    iputils-ping \
    curl \
    wget \
    nano \
    iftop \
    tcptrack \
    wondershaper \
    cron
```

Rôle : Pare-feu principal avec filtrage de paquets (iptables), proxy web (Squid), filtrage de contenu (SquidGuard) et outils de monitoring réseau.

1.3 Serveur Web DMZ (`ubuntu_webserver`)

Cette image héberge le serveur web situé dans la DMZ et intègre des outils de supervision et de sécurité :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    apache2 \
    lynx \
    iputils-ping \
    traceroute \
    curl \
    wget \
    nano \
    openssh-client \
```

```
    iproute2 \
    dnsutils \
    apache2-utils \
    tcpdump \
    net-tools \
    snort
```

Rôle : Serveur web Apache situé dans la DMZ, accessible depuis Internet mais isolé du réseau LAN interne. Il intègre également des outils de supervision (`tcpdump`, `net-tools`) et un IDS (`snort`) pour la détection d'intrusions.

1.4 Serveur Web LAN (`ubuntu_lan_webserver`)

Cette image simule un serveur web interne au LAN :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    apache2 \
    lynx \
    iputils-ping \
    traceroute \
    curl \
    wget \
    nano \
    openssh-client \
    iproute2 \
    dnsutils \
    apache2-utils
```

Rôle : Serveur web interne au LAN, utilisé pour tester l'accès et l'isolation entre le LAN et la DMZ/WAN.

1.5 WAN (`ubuntu_wan`)

Cette image simule une machine externe sur Internet :

```
FROM ubuntu:20.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    hping3 \
    lynx \
    iputils-ping \
    traceroute \
    curl \
```

```
    wget \
    nano \
    openssh-client \
    iproute2 \
    apache2 \
    dnsutils
```

Rôle : Machine externe utilisée pour simuler des connexions depuis Internet et tester les règles de sécurité.

Toutes ces images utilisent Ubuntu 20.04 comme base et incluent la variable d'environnement `DEBIAN_FRONTEND=noninteractive` pour automatiser l'installation des paquets sans interaction utilisateur.

2. **Mise en place de la topologie**

2.1 Création des containers dockers dans gns3

firewall

- docker image : ubuntu_firewall
- interfaces : 3
- console : telnet

public-server

- docker image : ubuntu_webserver
- interfaces : 1
- console : telnet

internal-server

- docker image : ubuntu_lan_webserver
- interfaces : 1
- console : telnet

internal-desktop

- docker image : ubuntu_client
- interfaces : 1
- console : telnet

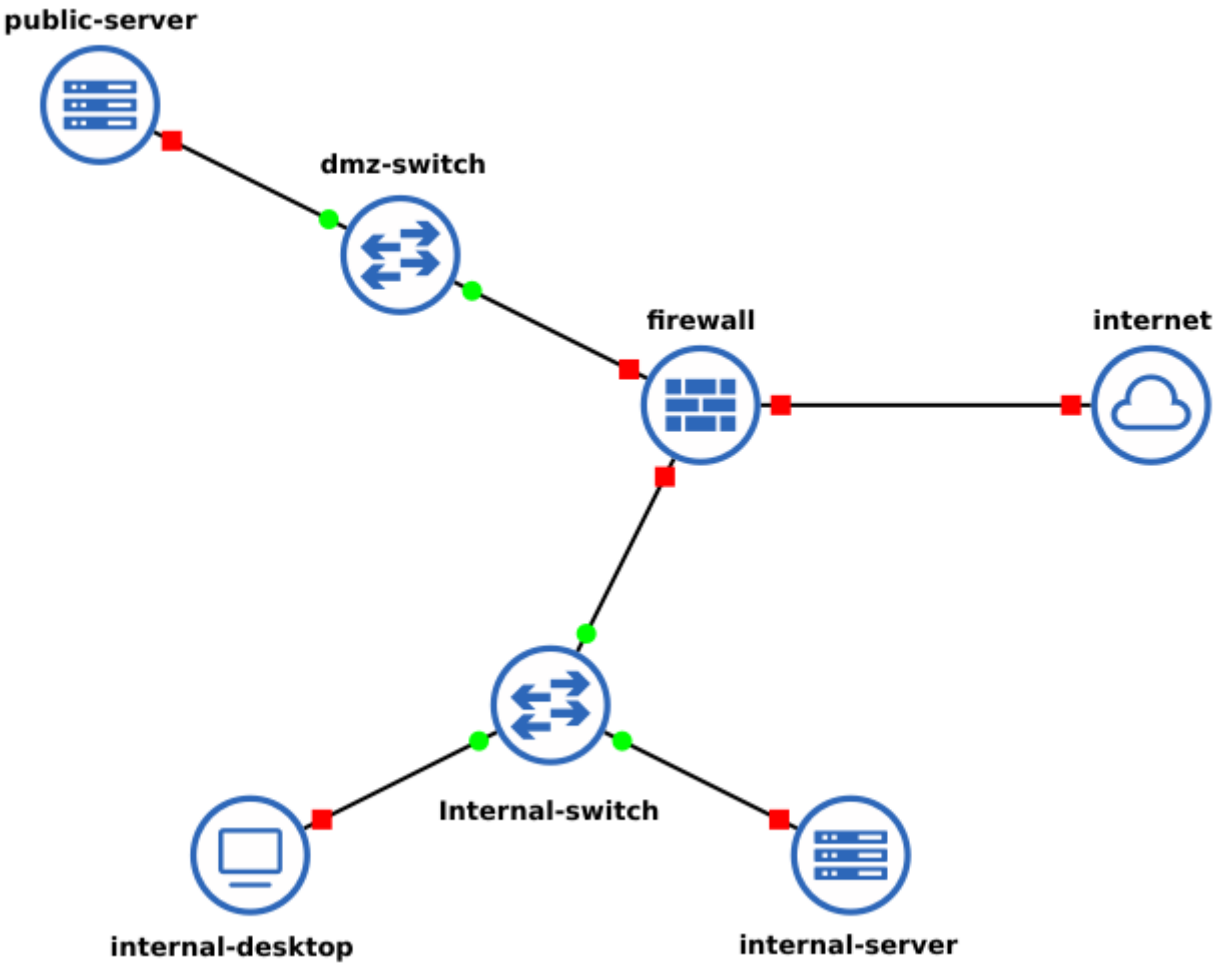
internet

- docker image : ubuntu_wan
- interfaces : 1
- console : telnet

2.2 Mise en place de la topologie dans gns3

1. Instanciation et liaison des containers

On commence par instancier les containers et les relier comme dans la figure ci-dessous :



Structure réseau :

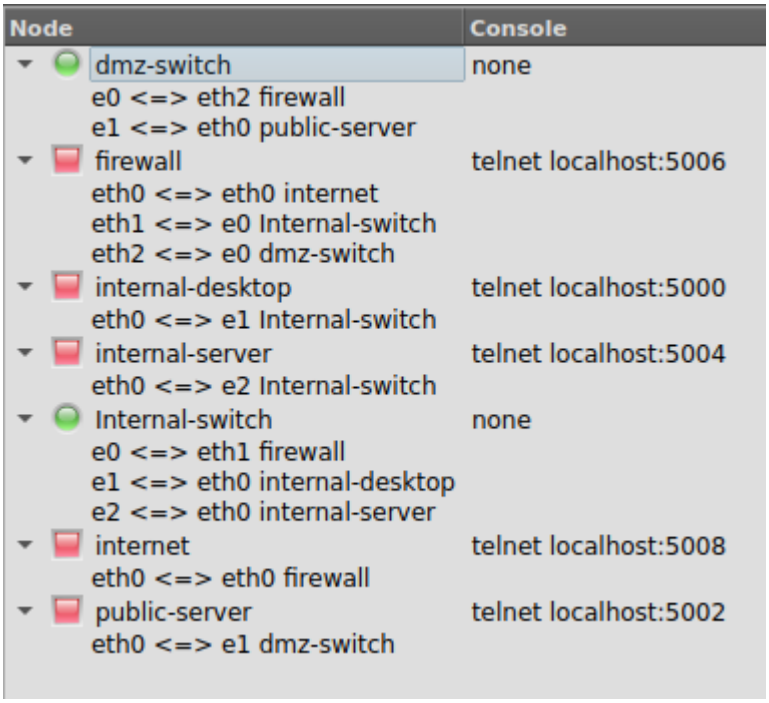
- **dmz-switch**
  - e0 ←→ eth2 firewall
  - e1 ←→ eth0 public-server
- **firewall**
  - eth0 ←→ e0 internal-switch
  - eth1 ←→ e0 internal-switch
  - eth2 ←→ e1 dmz-switch
- **internal-desktop**
  - eth0 ←→ e1 internal-switch
- **internal-server**
  - eth0 ←→ e2 internal-switch
- **internal-switch**

    - e0 ←→ eth1 firewall
    - e1 ←→ eth0 internal-desktop
    - e2 ←→ eth0 internal-server

- **internet**
    - eth0 ←→ eth0 firewall

- **public-server**
    - eth0 ←→ e1 dmz-switch

Image de la liaison des noeuds :



2. Démarrage et configuration des adresses IP

Après le démarrage de toutes les instances, on configure leurs adresses IP comme suit :

a) LAN

- **internal-desktop**

```
ip a add 19.4.3.1/29 dev eth0
ip route add default via 19.4.3.3 dev eth0
```

- **internal-server**

```
ip a add 19.4.3.2/29 dev eth0
ip route add default via 19.4.3.3 dev eth0
```

b) WAN

- **internet**

```
ip a add 19.4.3.17/29 dev eth0
ip route add default via 19.4.3.18 dev eth0
```

c) DMZ

- **public-server (DMZ)**

```
ip a add 19.4.3.9/29 dev eth0
ip route add default via 19.4.3.11 dev eth0
```

d) Pare-feu

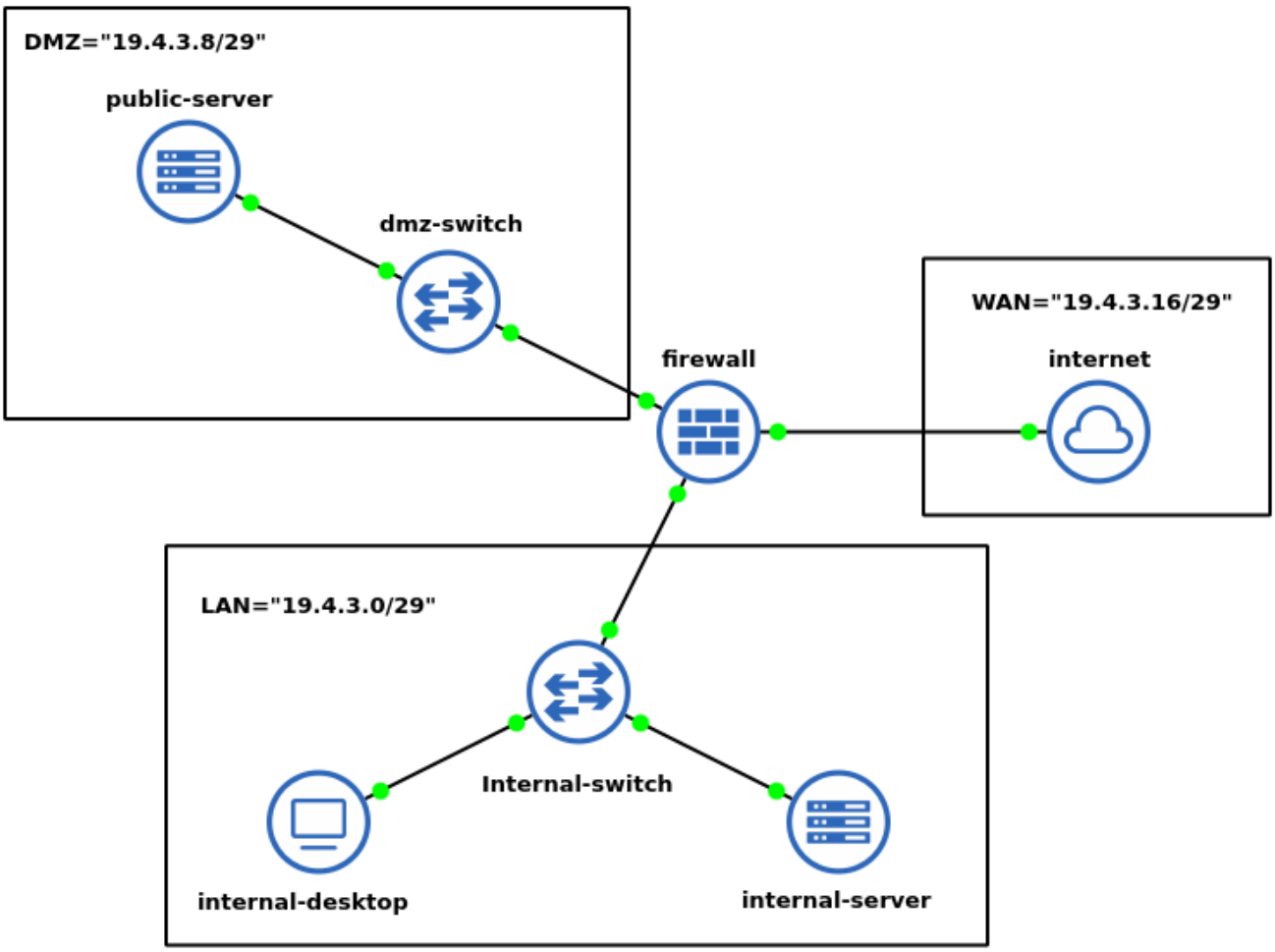
- **firewall**

```
ip a add 19.4.3.3/29 dev eth1
ip a add 19.4.3.11/29 dev eth2
ip a add 19.4.3.18/29 dev eth0
sysctl -w net.ipv4.ip_forward=1
```

Subnets utilisés

```
WAN="19.4.3.16/29"
LAN="19.4.3.0/29"
DMZ="19.4.3.8/29"
```

Image de la topologie :

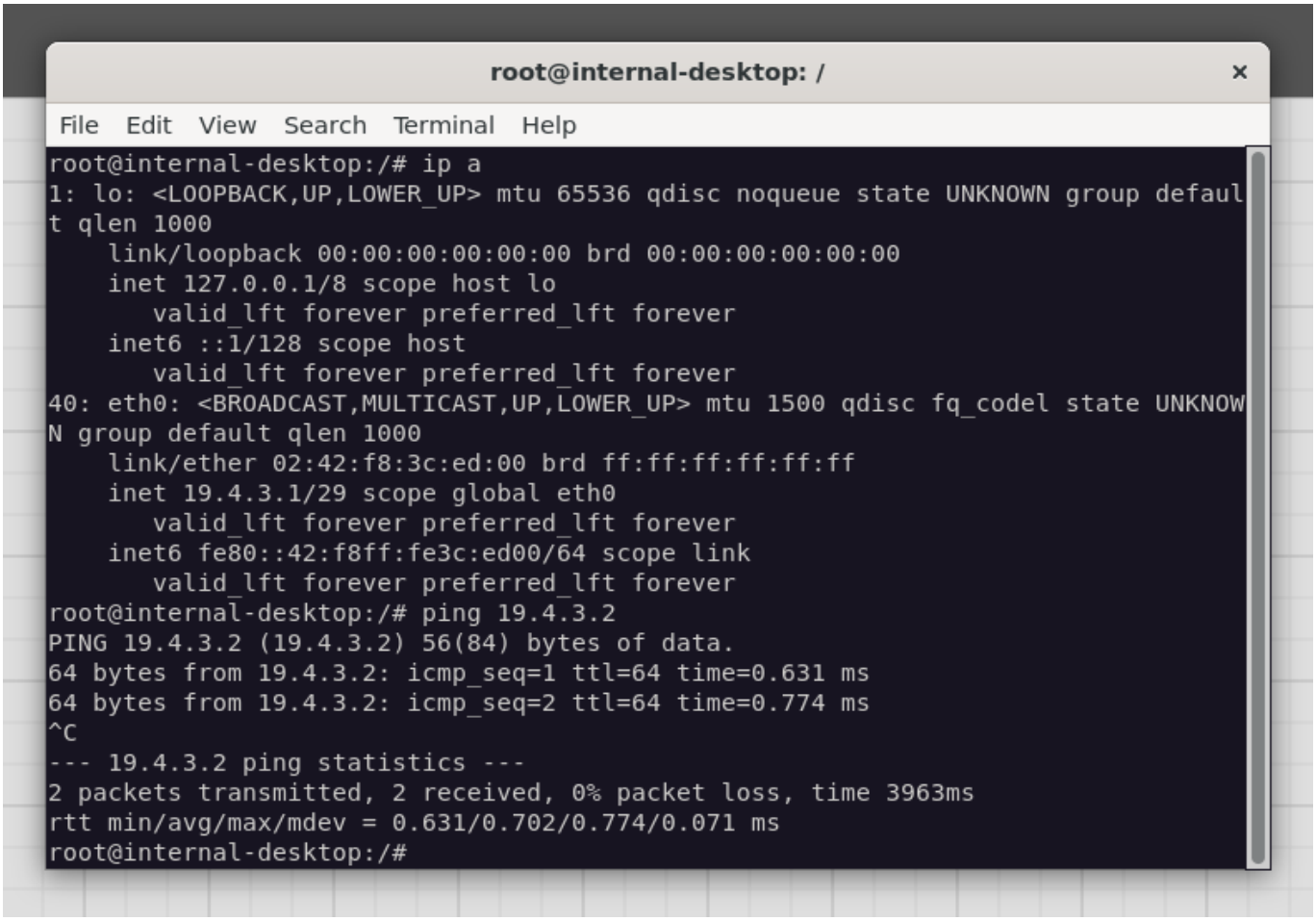


3. Test de la topologie de base

1. LAN -> LAN

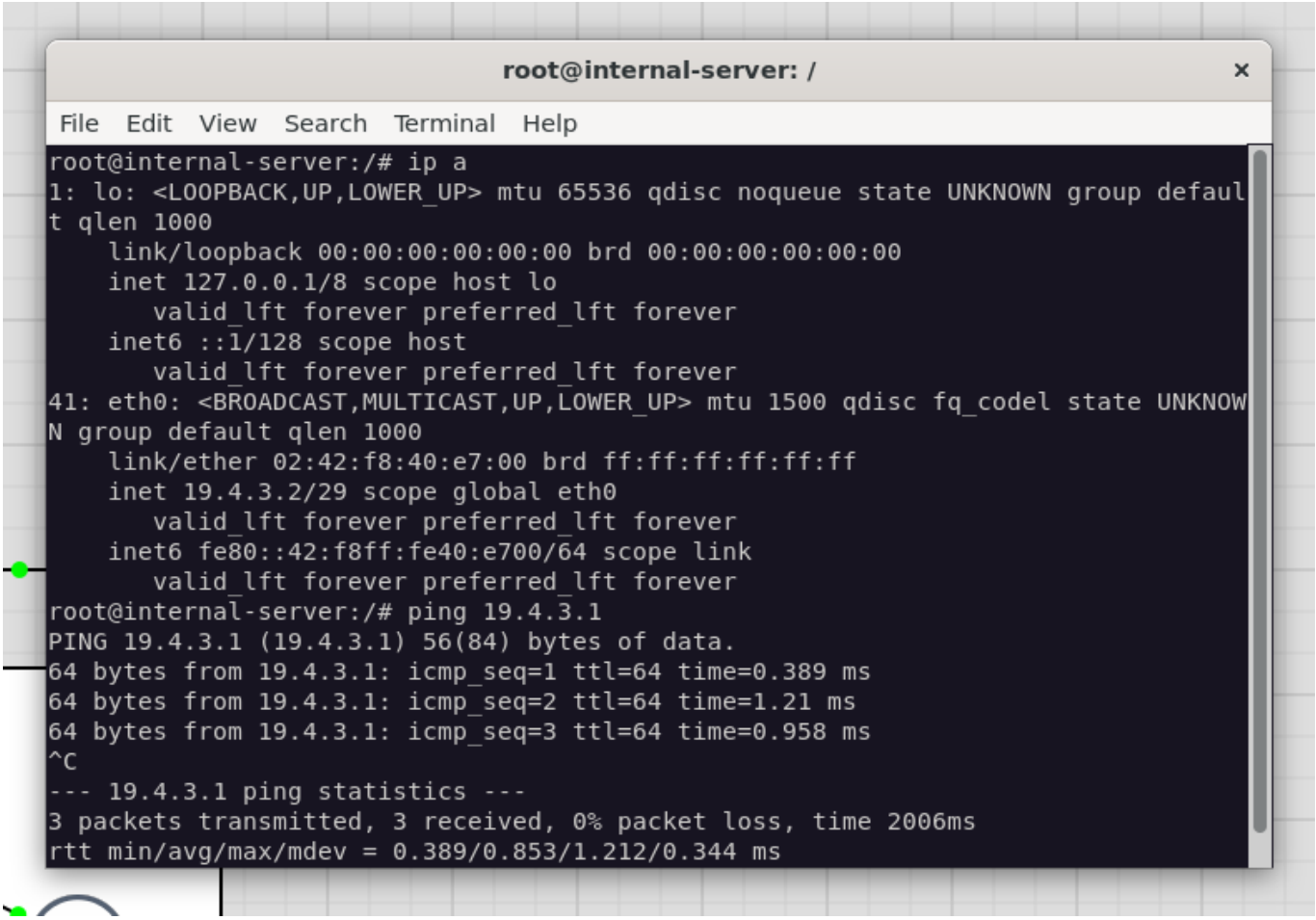
- **internal-desktop -> internal-server**

```
ping 19.4.3.2
```



- **internal-server -> internal-desktop**

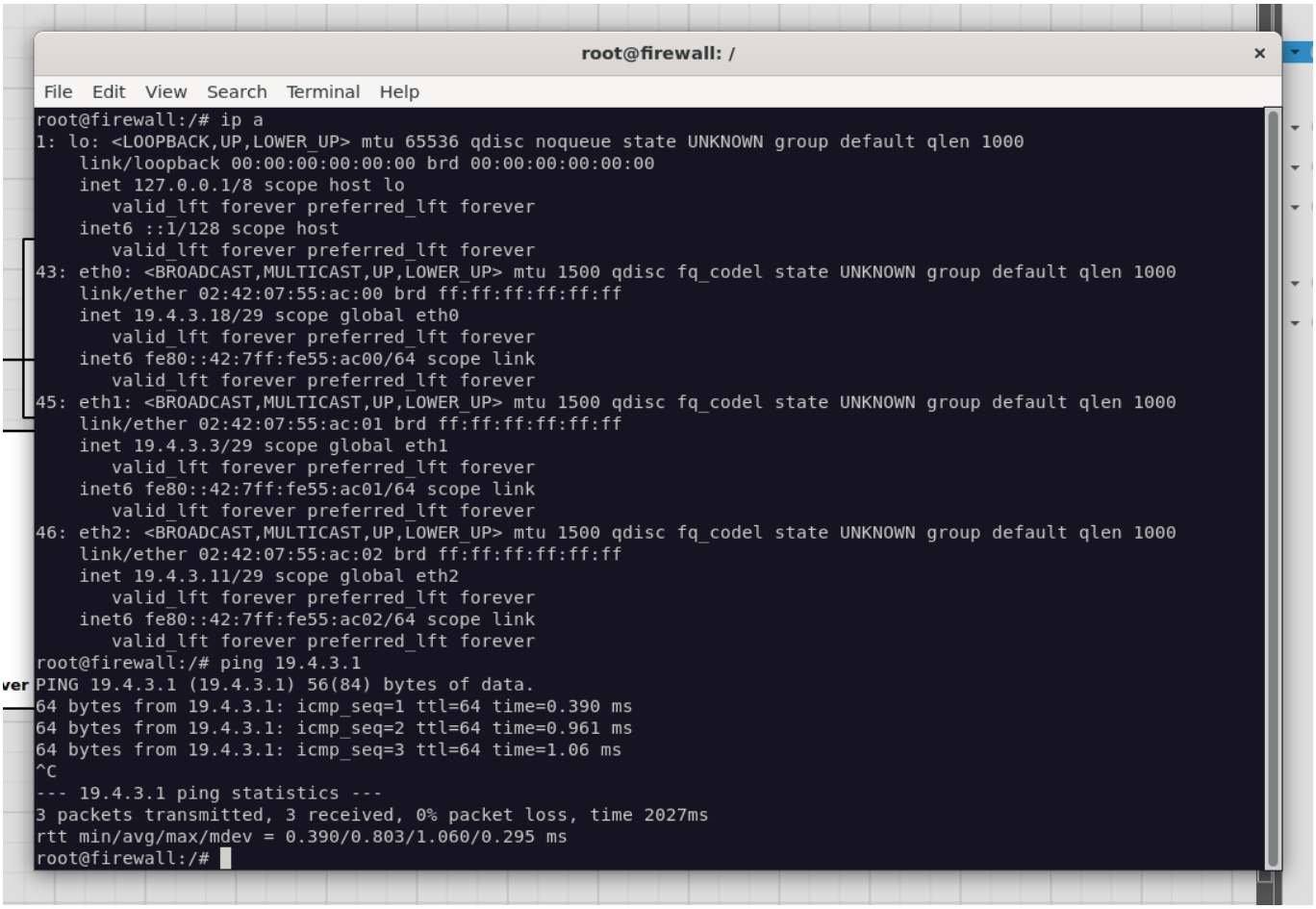
```
ping 19.4.3.1
```



2. firewall <-> LAN

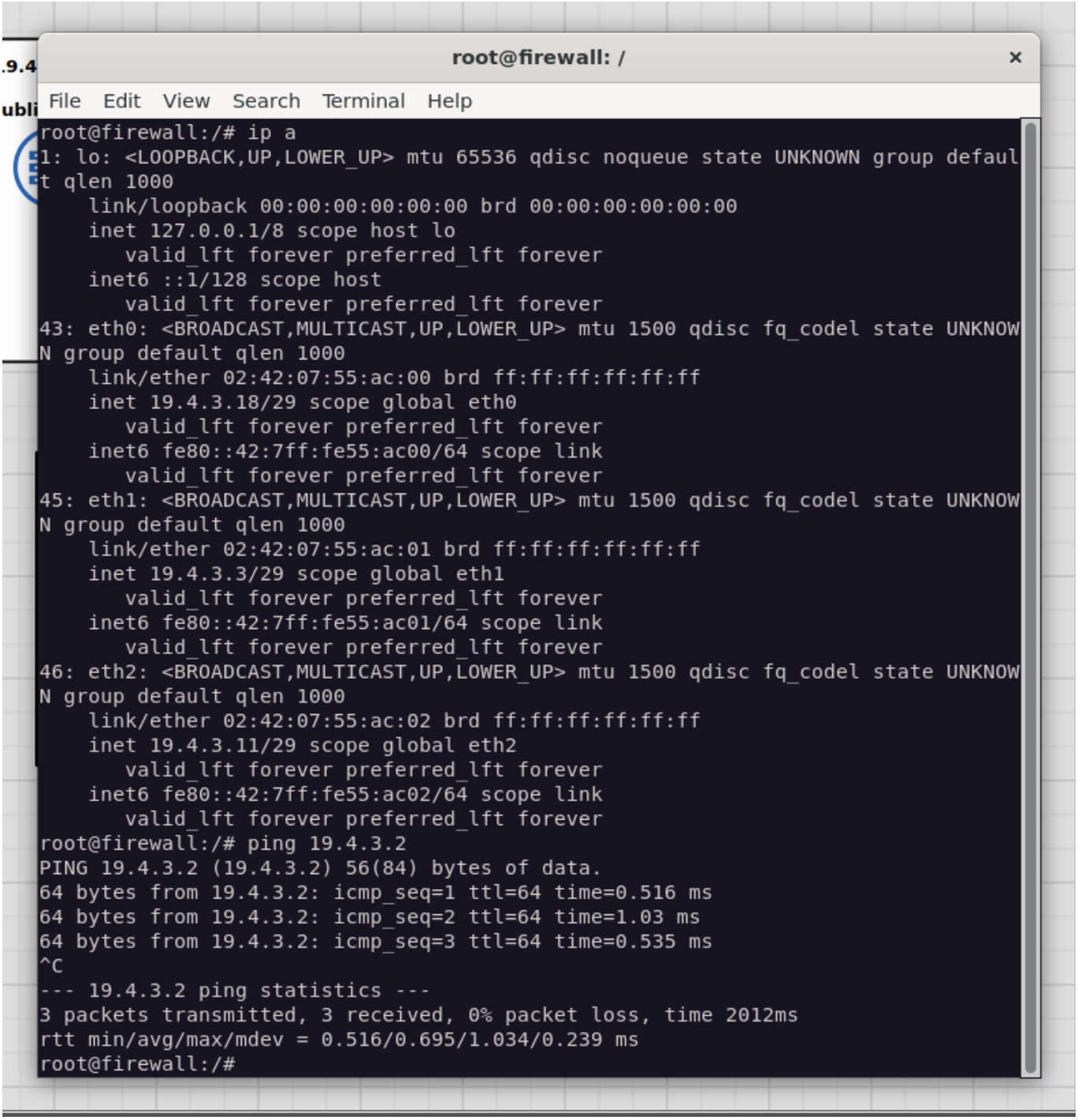
- **firewall -> internal-desktop**

```
ping 19.4.3.1
```



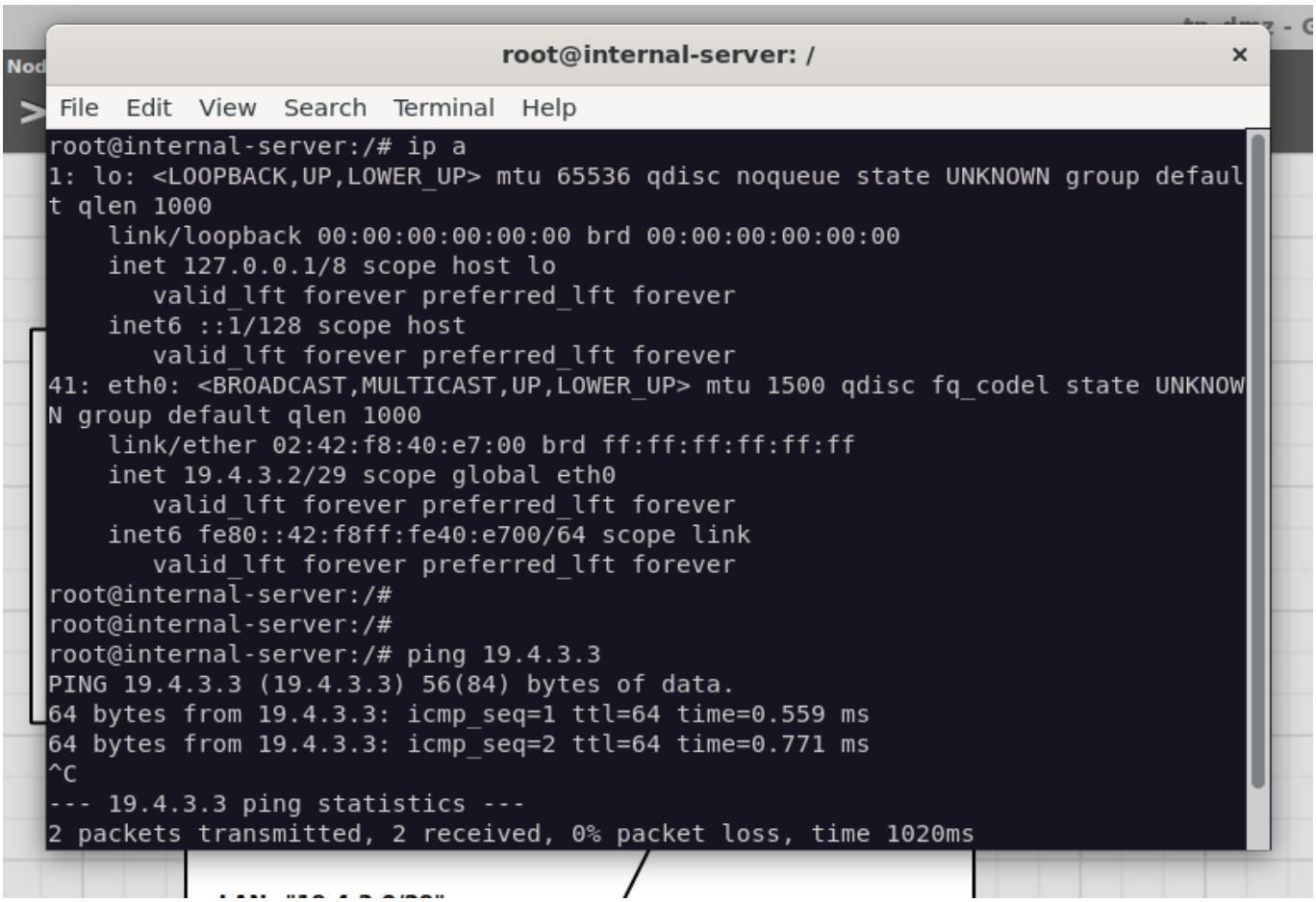
- **firewall -> internal-server**

```
ping 19.4.3.2
```



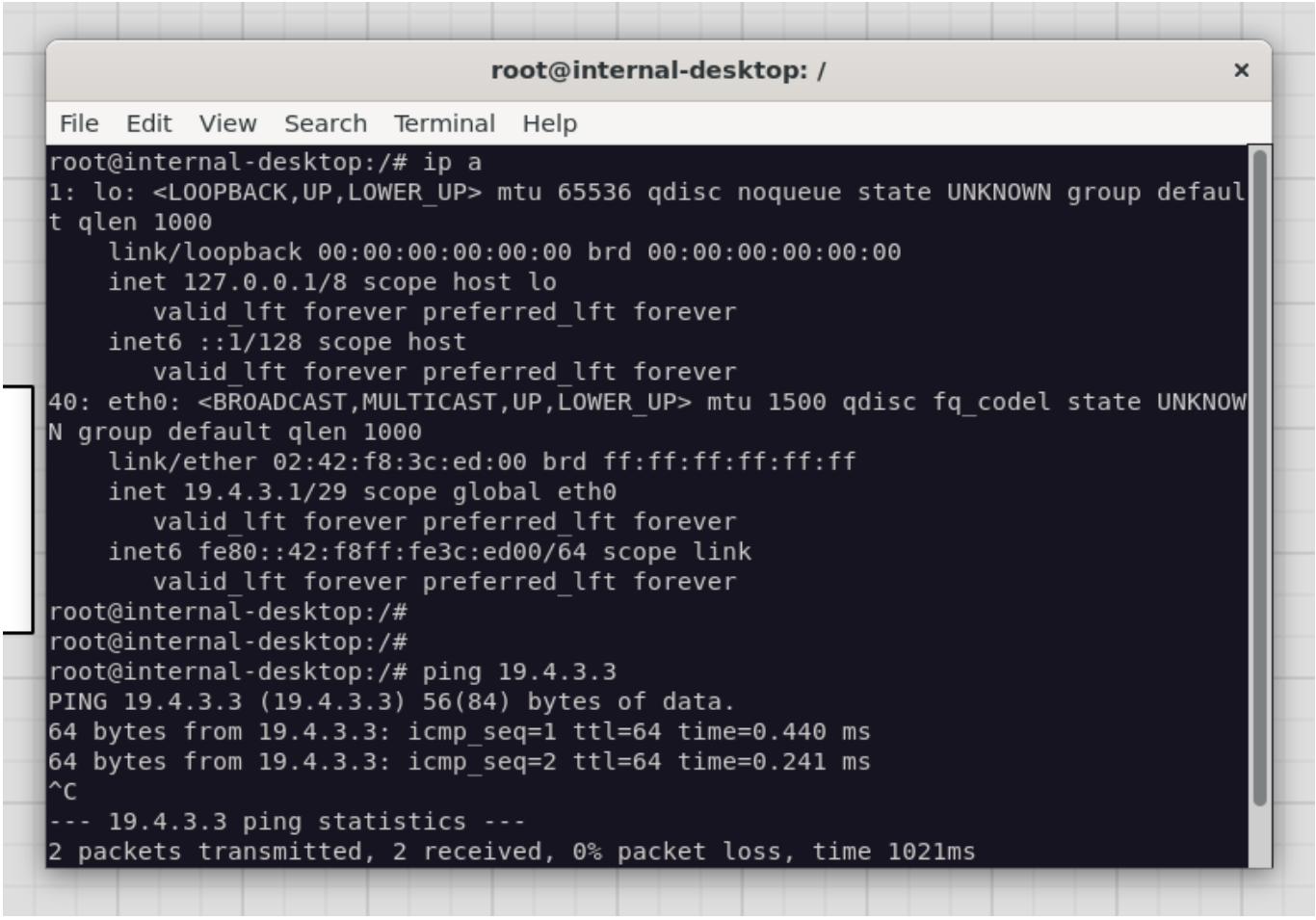
- **internal-server -> firewall**

```
ping 19.4.3.3
```



- **internal-desktop -> firewall**

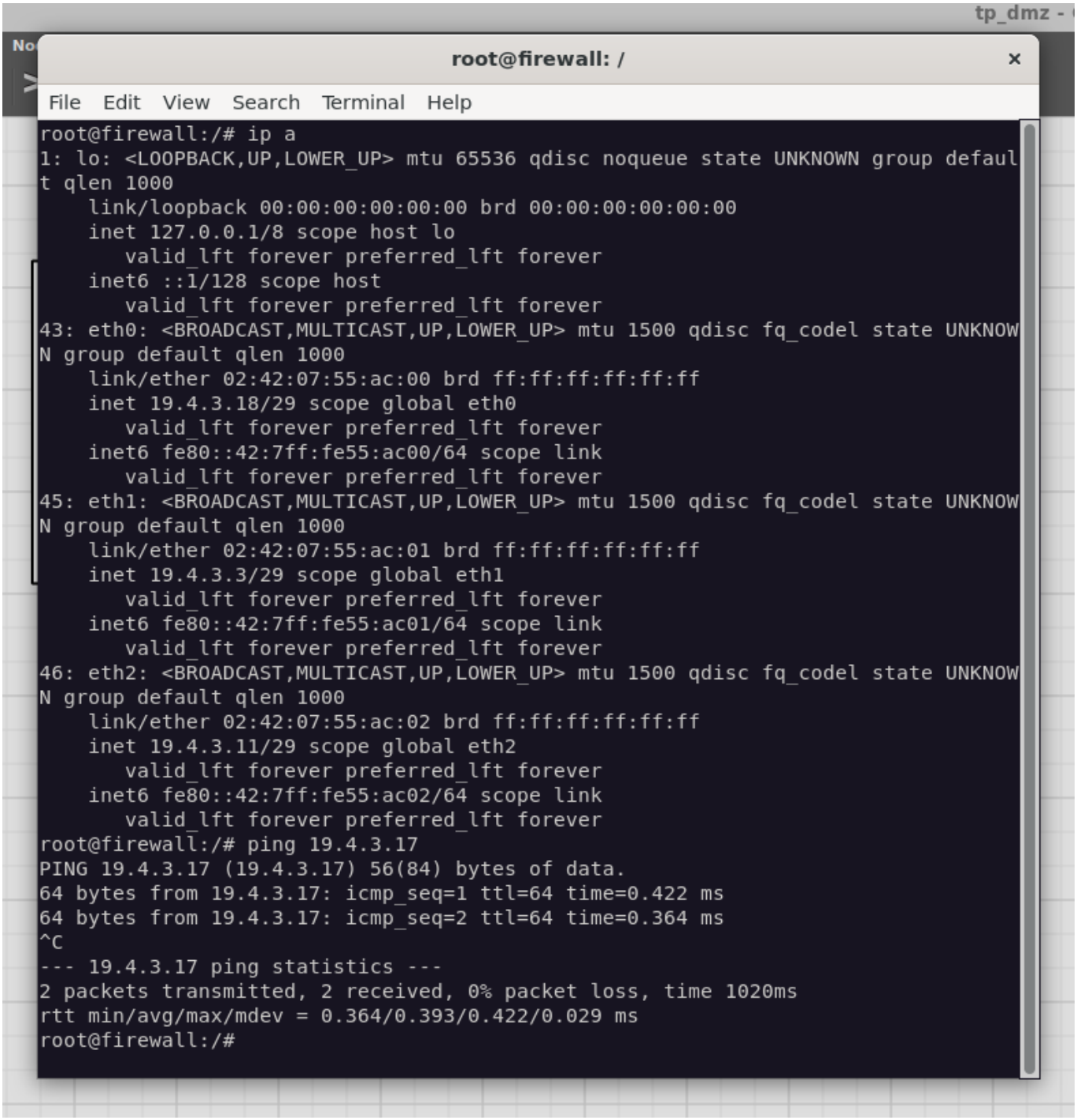
```
ping 19.4.3.3
```



3. firewall <-> WAN

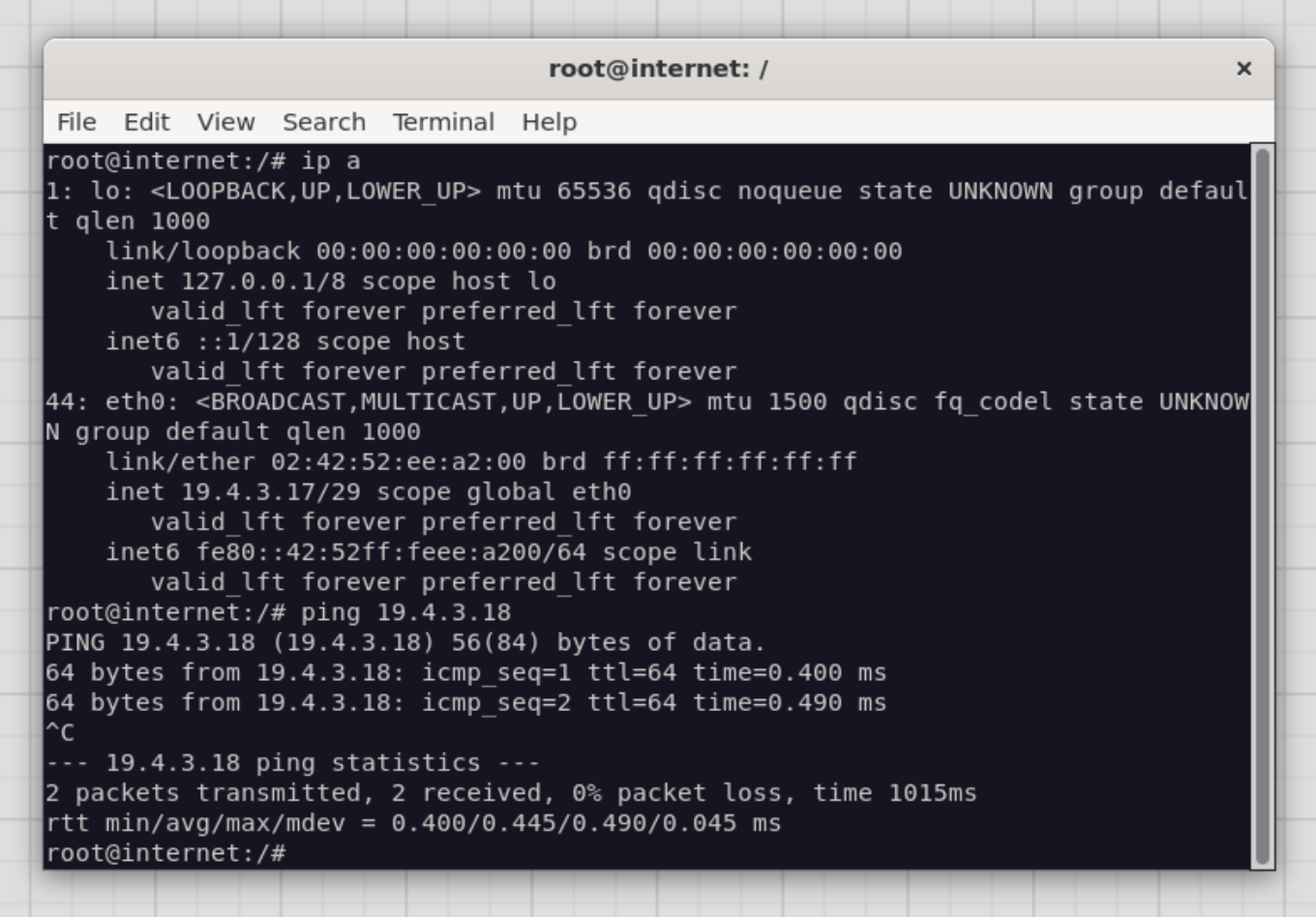
- **firewall -> internet**

```
ping 19.4.3.17
```



- **internet -> firewall**

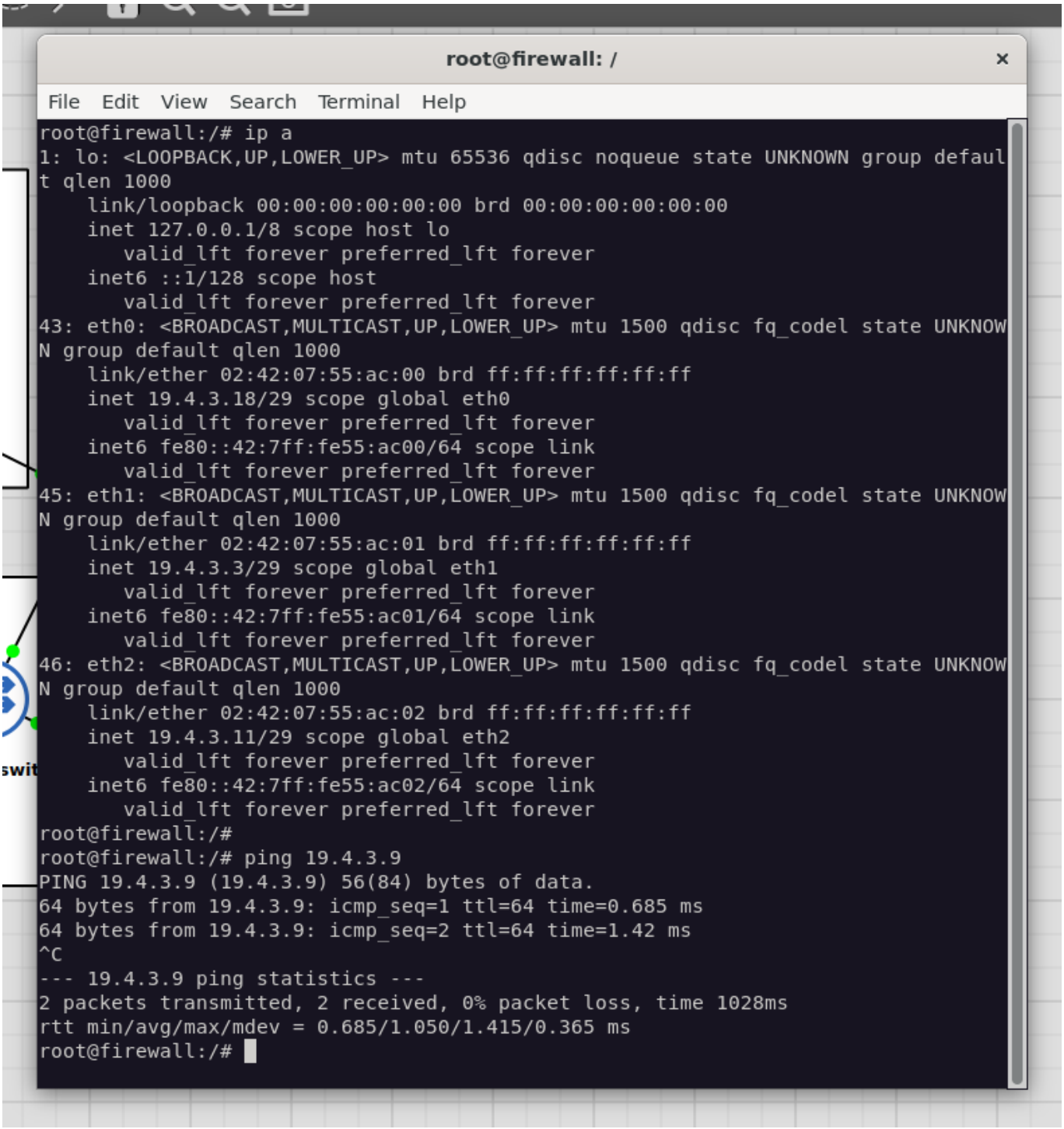
```
ping 19.4.3.18
```



4. DMZ <-> firewall

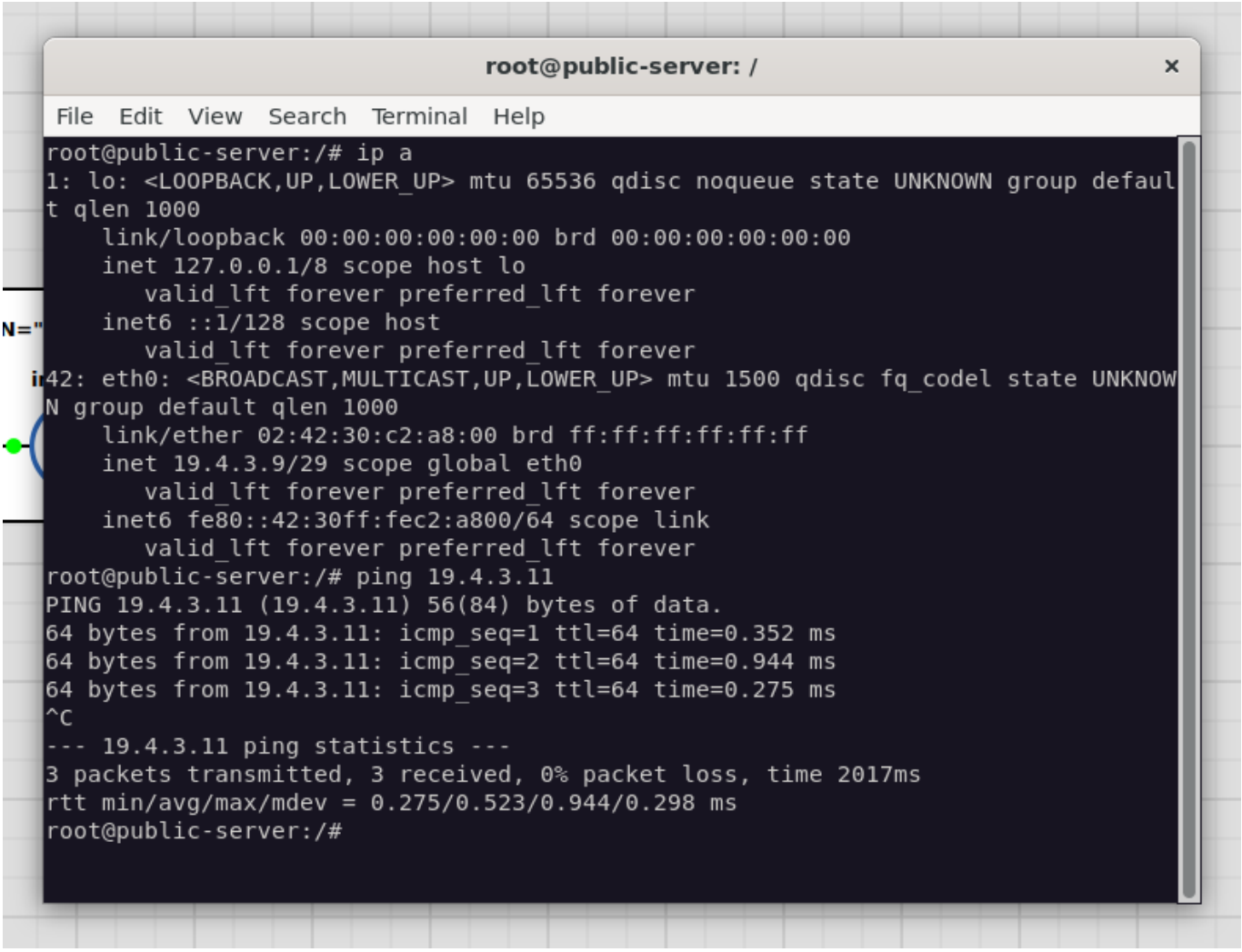
- **firewall -> DMZ**

```
ping 19.4.3.9
```



- **DMZ -> firewall**

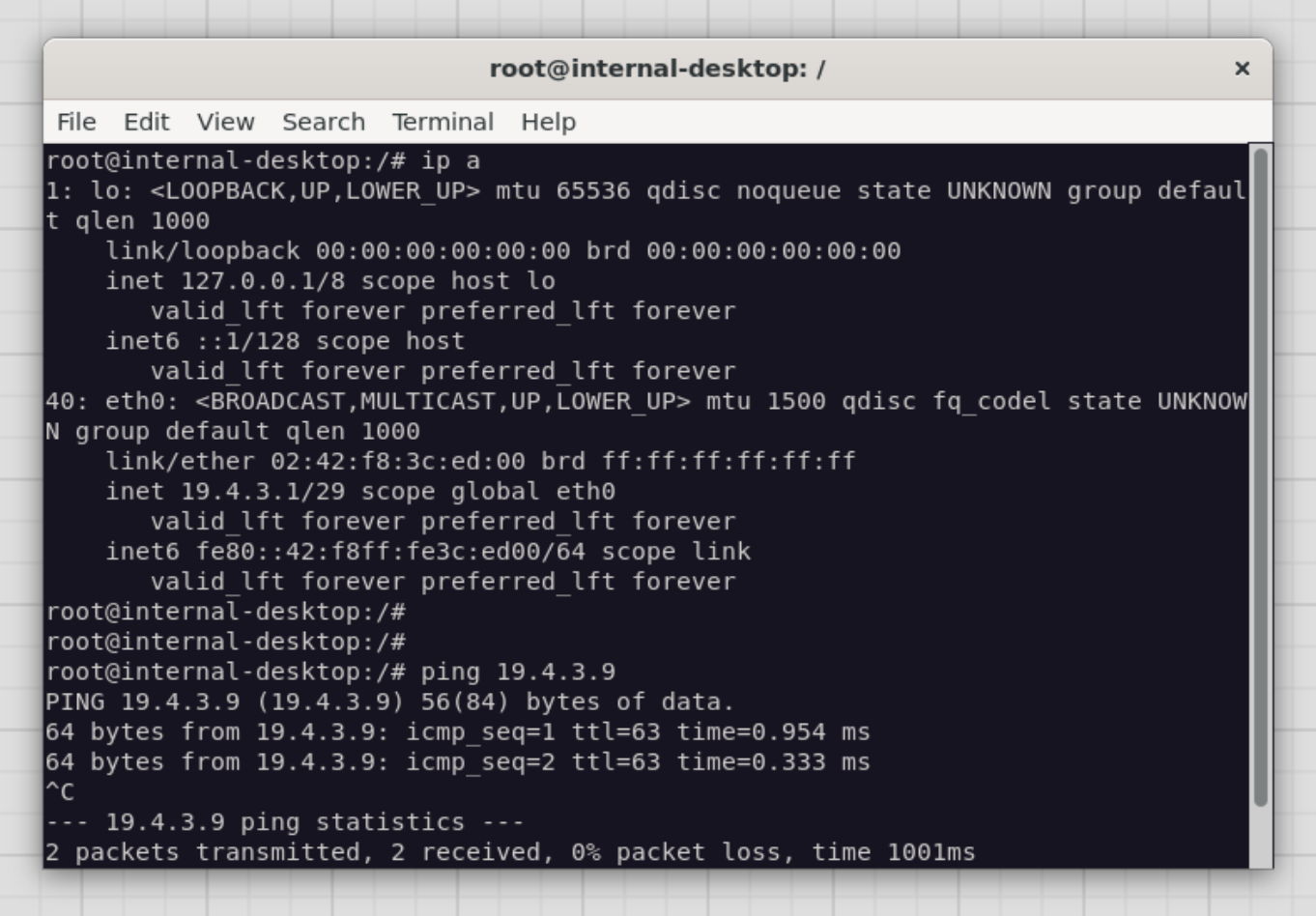
```
ping 19.4.3.11
```



5. LAN <-> DMZ

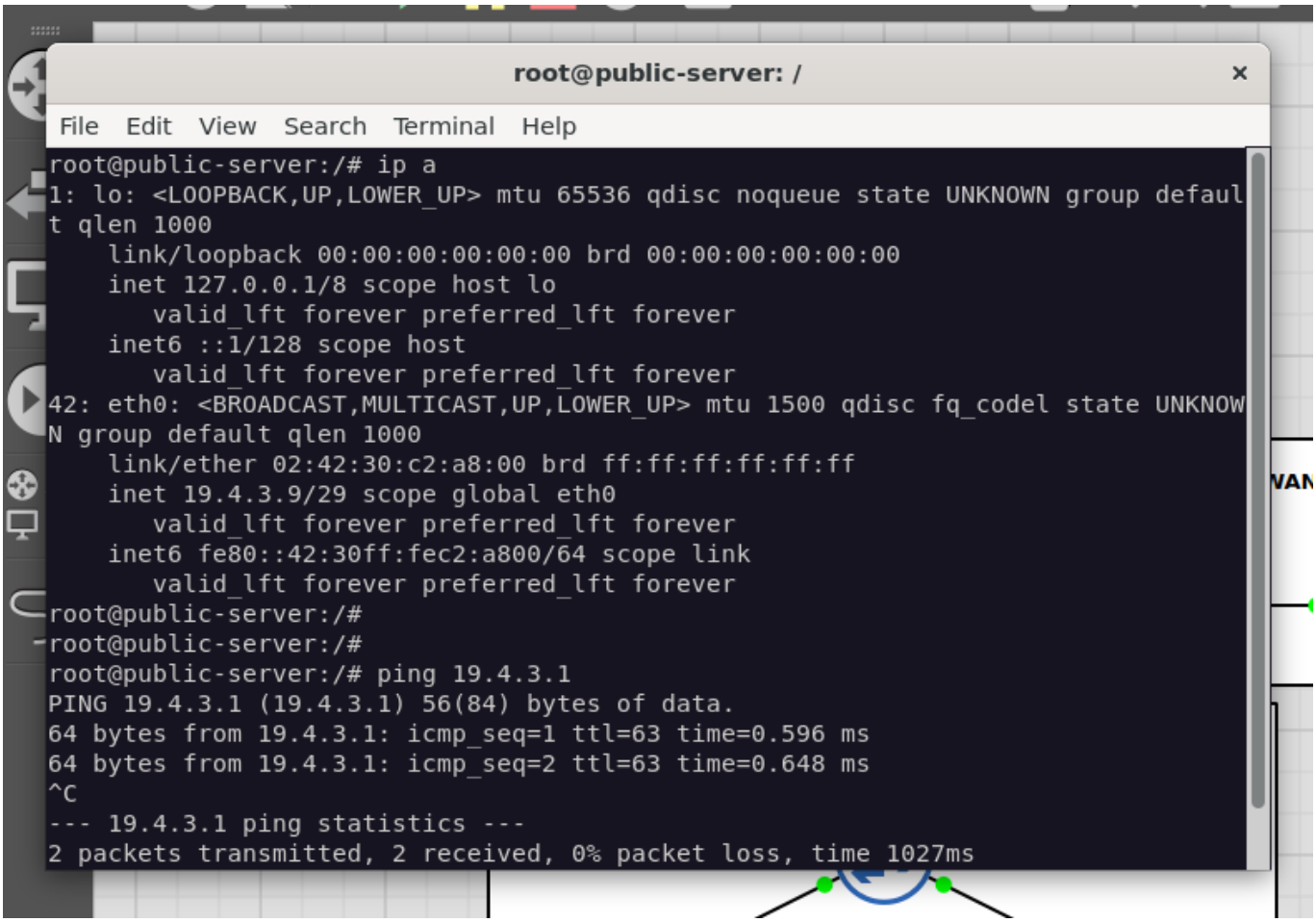
- **LAN -> DMZ**

```
ping 19.4.3.9
```



- **DMZ -> LAN**

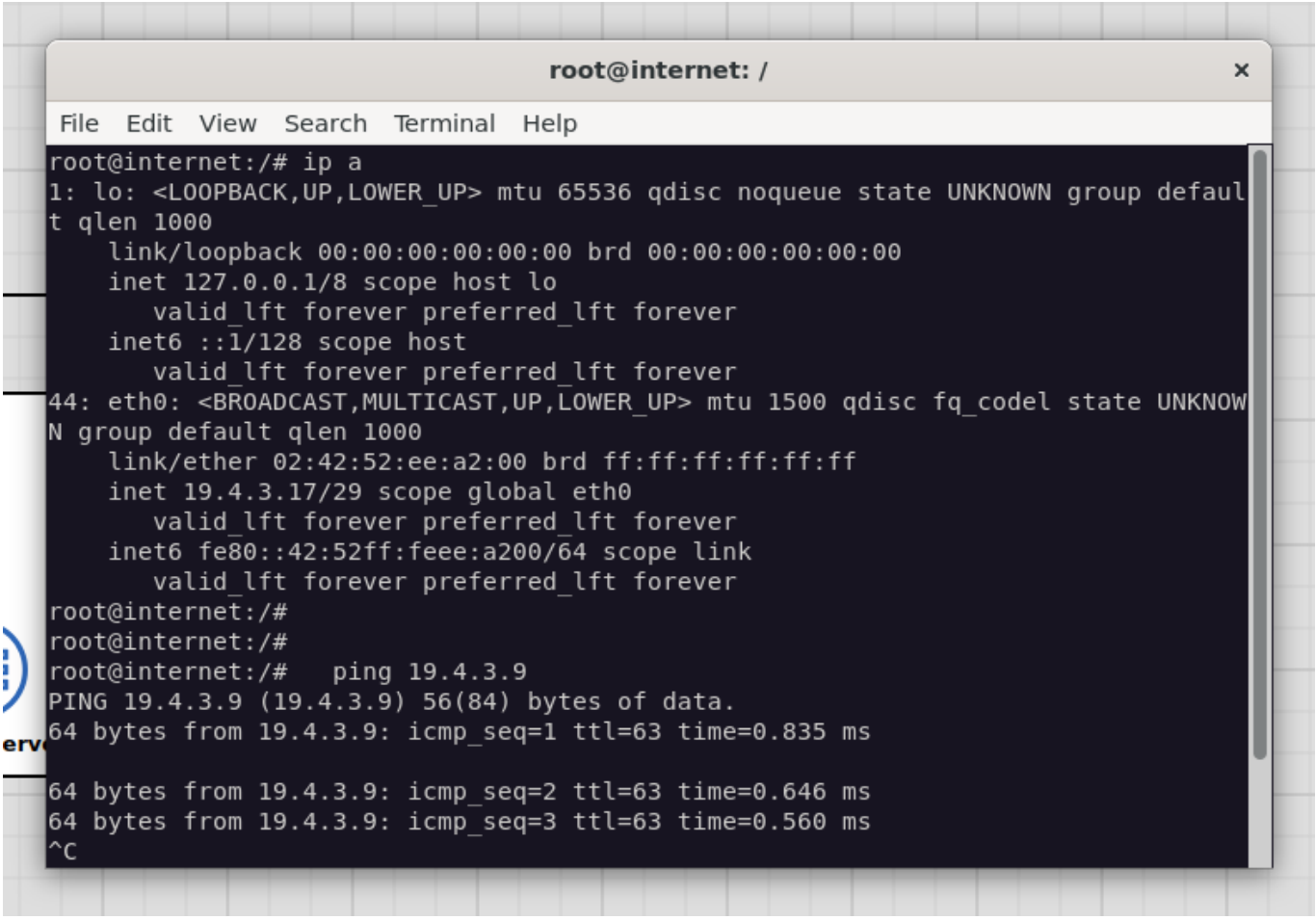
```
ping 19.4.3.1
```



6. WAN -> DMZ

- **LAN -> DMZ**

```
ping 19.4.3.9
```


root@internet: /
File Edit View Search Terminal Help
root@internet:/# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defaul
t qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host
valid_lft forever preferred_lft forever
44: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOW
N group default qlen 1000
link/ether 02:42:52:ee:a2:00 brd ff:ff:ff:ff:ff:ff
inet 19.4.3.17/29 scope global eth0
valid_lft forever preferred_lft forever
inet6 fe80::42:52ff:feee:a200/64 scope link
valid_lft forever preferred_lft forever
root@internet:/#
root@internet:/#
root@internet:/# ping 19.4.3.9
PING 19.4.3.9 (19.4.3.9) 56(84) bytes of data.
64 bytes from 19.4.3.9: icmp_seq=1 ttl=63 time=0.835 ms
64 bytes from 19.4.3.9: icmp_seq=2 ttl=63 time=0.646 ms
64 bytes from 19.4.3.9: icmp_seq=3 ttl=63 time=0.560 ms
^C